

Topic: Types of Queue – Circular Queue, Priority Queue

Q. Explain the limitations of a simple linear queue. How do a Circular Queue and a Priority Queue overcome specific challenges? Describe their working principles.

📌 Definition to Remember

A **Circular Queue** connects the last array index back to the first, solving the memory wastage problem of linear queues. A **Priority Queue** dequeues elements based on their assigned **priority value** rather than strict arrival time (FIFO), allowing critical tasks to be processed first.

1. Limitation of Linear Queue

In a standard linear array queue, once the **rear** pointer reaches the maximum index ($\text{SIZE} - 1$), no new elements can be enqueued. This happens **even if spaces are empty at the front** due to earlier dequeue operations, leading to severe **memory wastage**.

2. Circular Queue

Solution: The queue forms a logical circle. The next position after the last index is index 0.

- **Working Principle:** Uses the **modulo operator (%)** to wrap pointers around.
- Next **rear** = $(\text{rear} + 1) \% \text{SIZE}$
- Next **front** = $(\text{front} + 1) \% \text{SIZE}$
- **Queue Full Condition:** $(\text{rear} + 1) \% \text{SIZE} == \text{front}$
- **Advantage:** Efficiently reuses freed memory at the beginning of the array.
- **Applications:** Traffic light systems, Memory Management, CPU round-robin scheduling.

```

    [0] ← front
  /      \
[3]        [1]
  \      /
    [2] ← rear
(If SIZE=4, next rear after 3 is  $(3+1)\%4 = 0$ )
```

3. Priority Queue

Problem: A standard queue strictly processes the oldest element first. Some real-world tasks (like CPU interrupts) are urgent and cannot wait in a FIFO line.

Solution: Each element is assigned a **priority**.

- **Working Principle:**
- Elements are dequeued based on **highest priority**, not arrival time.
- If two elements have the *same* priority, they are served based on arrival (FIFO).
- **Implementation:** Best implemented using **Heaps** (Min-Heap or Max-Heap) which provide highly efficient $O(\log n)$ insertion and deletion times.
- **Applications:** CPU task scheduling (interrupt handling), Dijkstra's Shortest Path algorithm in graphs.

★ Must-Write Points (for 10 marks)

1. **Linear Queue Limitation:** Wastes memory; cannot insert when rear is at the end, even if front is empty.
2. **Circular Queue:** Connects the last index to the first using the modulo operator $(\text{rear} + 1) \% \text{SIZE}$.
3. Circular Queue perfectly reuses freed memory space. It is full when $(\text{rear} + 1) \% \text{SIZE} == \text{front}$.
4. **Priority Queue:** Elements are removed based on a **priority value**, not strict FIFO.
5. In Priority Queues, if priorities tie, FIFO determines the order.
6. Priority Queues are optimally implemented using **Heaps (Min/Max)** for $O(\log n)$ efficiency.
7. Circular Q used in round-robin scheduling; Priority Q used in CPU interrupt handling.

⚡ Quick Recall

Linear Q wastes memory → Circular Q wraps around $((\text{rear}+1)\% \text{SIZE})$, reuses space → Priority Q breaks FIFO for urgent tasks, served by highest priority → implemented via Heaps